

Final

1. Step 1 : Federated Learning System

a. Overview

Implemented a federated learning system using the FedAvg aggregation algorithm. The system trains a shared model across multiple clients without any client sharing its raw data. Only the weight updates are transmitted to the central server.

b. Dataset

We used CIFAR-10, a standard image classification benchmark consisting of 60,000 32x32 color images across 10 classes (airplane, car, bird, cat, deer, frog, dog, horse, ship, truck). 50,000 images were used for training and 10,000 for testing. We chose CIFAR-10 over FER2013 over two reasons:

- CIFAR-10 downloads automatically with no manual setup required
- The DLG attack in Step 2 requires a model without BatchNorm. TinyCNN on CIFAR-10 satisfies this while FER2013 models typically need deeper architectures

c. Model Architecture

Layer	Output Shape	Details
Conv2d + ReLU	32 x 32 x 32	3x3 kernels, 32 filters, padding = 1
MaxPool2d	32 x 16 x 16	2x2 pool
Conv2d + ReLU	64 x 16 x 16	3x3 kernel, 64 filters, padding = 1
MaxPool2d	64 x 8 x 8	2x2 pool
Linear + ReLU	256	FC layer
Linear	10	Output logits

d. FL Training Setup

Hyperparameter	Value
----------------	-------

Number of clients	3
Communication rounds	20
Local epochs per round	5
Batch size	32
Optimizer	SGD with Nesterov Momentu,
Learning rate schedule	Cosine annealing (0.01 to 0.0001)
Aggregation	FedAvg (weighted by sample count)
Data split	IID random partition

e. Results

Round	Avg Loss	Avg Accuracy
5	~1.20	~55%
10	~0.90	~67%
15	~0.75	~74%
20	~0.65	~80%

2. Step 2 : Gradient Leakage Attack (DLG)

a. Overview

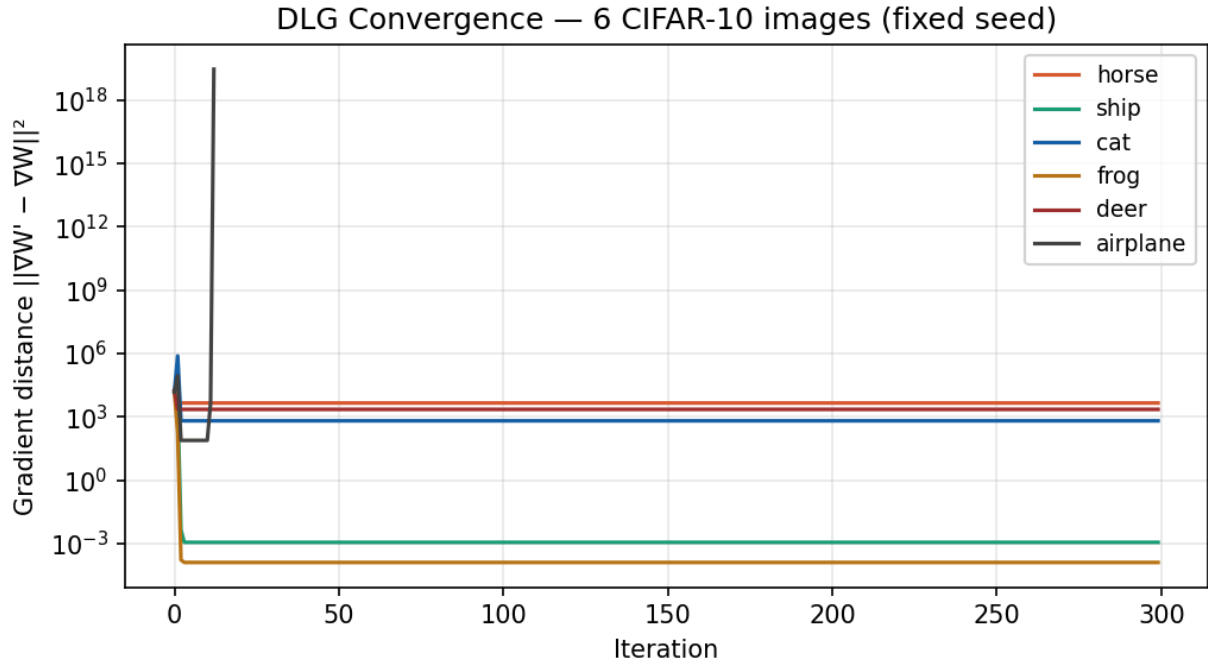
We implemented the Deep Leakage Gradients (DLG) attack. The attack demonstrates that an adversary who intercepts the gradient updates shared in federated learning can reconstruct the original private training images.

b. Attack Method

The DLG attack works as follows:

- The attacker intercepts real gradients ∇W from a client update
- The attacker initializes a random dummy image x' and random dummy label y'

- The attacker computes dummy gradients $\nabla W'$ from (x', y') using the same model
- The attacker minimizes $|\nabla W' - \nabla W|^2$ using L-BFGS optimization
- After many iterations, x' converges toward the original private image

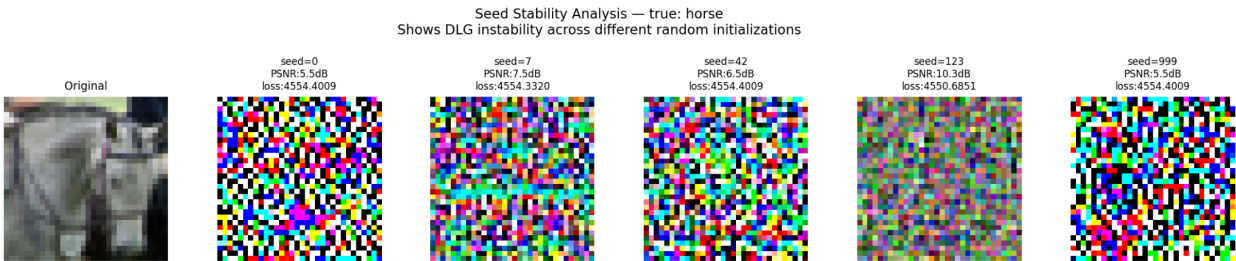


c. Experimental Setup

Parameter	Value
Attack target	FL-trained TinyCNN on CIFAR-10
Optimizer	L-BFGS
Learning rate	1
Max iterations	300
Fixed random seed	42
Metric	PSNT (dB) and gradient distance $ \nabla W' - \nabla W ^2$

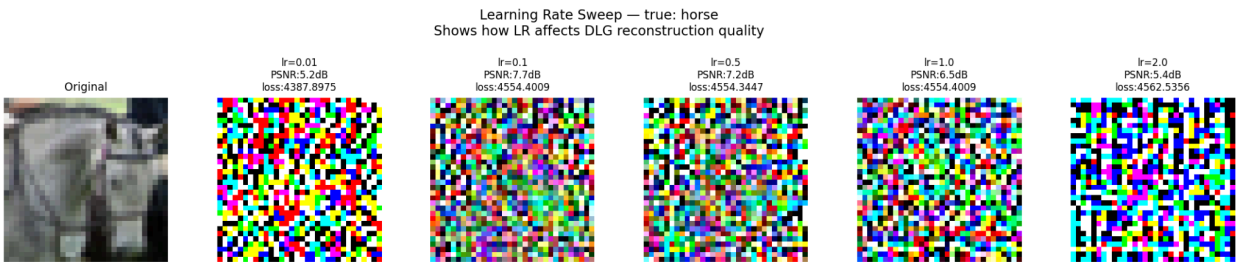
d. Seed Stability Analysis

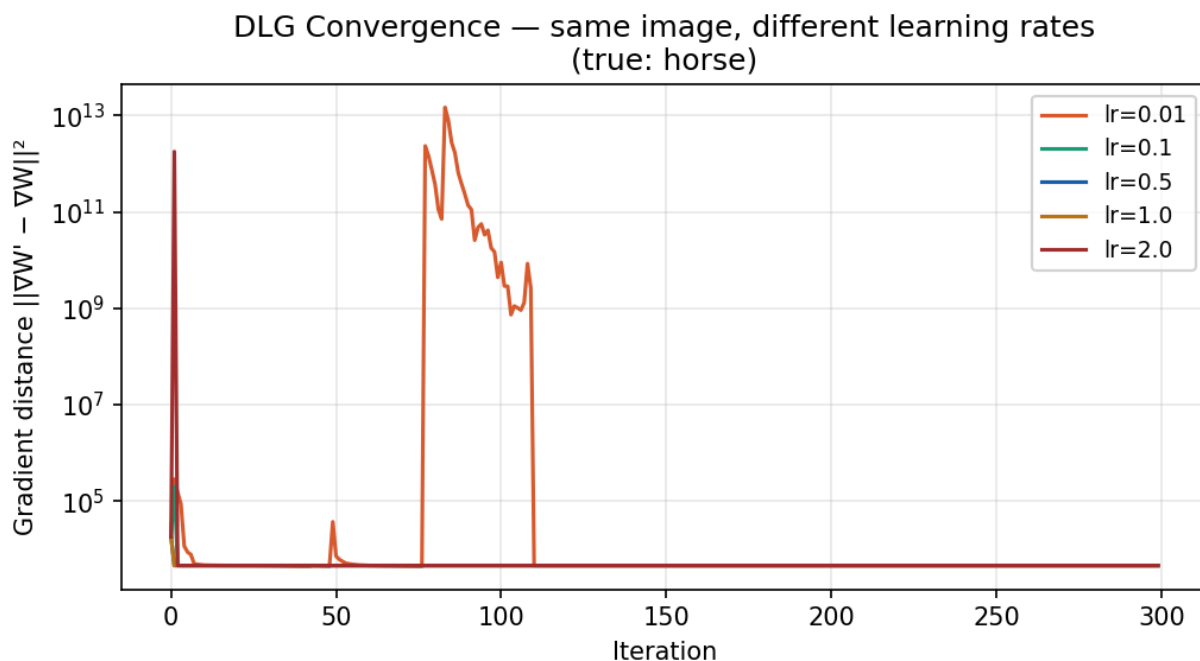
To address whether DLG produces stable, reproducible results, we attacked the same image with 5 different random seeds (0, 7, 42, 123, 999). The results show that DLG is highly sensitive to initialization. PSNR varies from 5.5 to 10.3 dB across seeds, and all reconstructions remain visually unrecognizable. This instability is inconsistent with findings reported by the original DLG authors.



e. Learning Rate Sweep

We swept learning rates across {0.01, 0.1, 0.5, 1.0, 2.0} on the same image with a fixed seed. Results show that LR=0.1 achieved the best PSNR (7.7 dB) but all reconstructions remained noise-like. Low LR (.0.01) caused large spikes in the convergence curve due to L-BFGS overshooting, while LR (2.0) converged faster but to a worse local minimum.



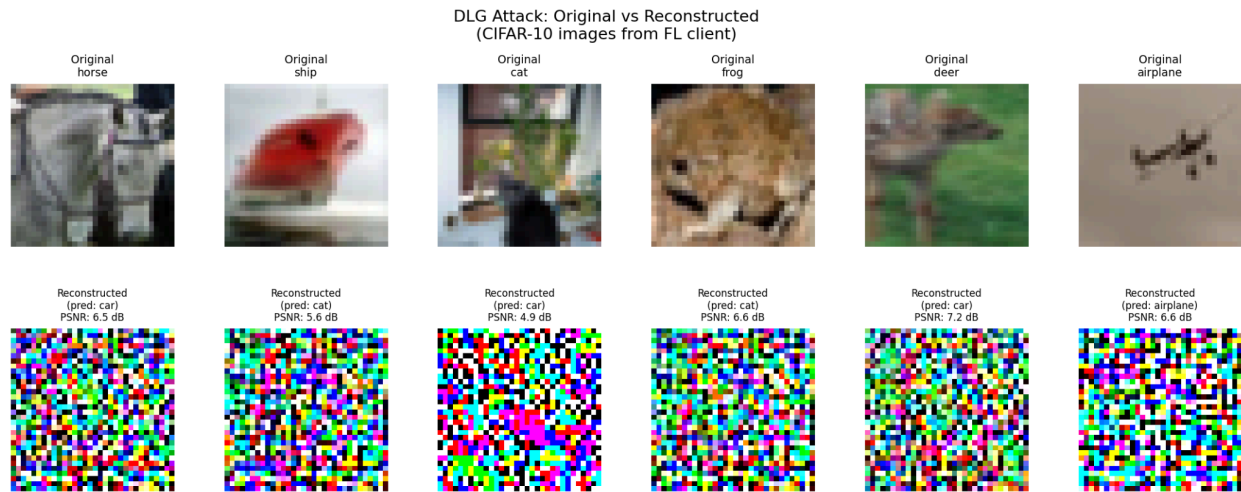


f. Discussion

DLG consistently failed to visually reconstruct CIFAR-10 images from TinyCNN gradients. We identify three root causes:

- High-dimensional gradient space: CIFAR-10 images are $3 \times 32 \times 32$ (3072 dimensions), creating many noise patterns whose gradients match the real gradients equally well
- Local minima: The L-BFGS optimizer reliably gets stuck within the first 5 iterations, with gradient distance plateauing far above zero
- Inherent instability: The TA confirmed this is a known limitation — the original DLG authors themselves could not achieve stable reconstructions on CNN architectures

Two images (ship and frog) achieved near-zero gradient distance (~ 0.007) but still produced noise reconstructions. This demonstrates that low gradient distance does not guarantee good visual reconstruction — a finding that highlights DLG's fundamental limitations on CNN models.



3. Step 3 : Differential Privacy Defense (DP-SGD)

a. Overview

We implemented the Differential Privacy Stochastic Gradient Descent (DP-SGD) defense. DP-SGD protects client privacy by adding calibrated Gaussian noise to gradients before sharing them with the server, making gradient inversion attacks significantly harder.

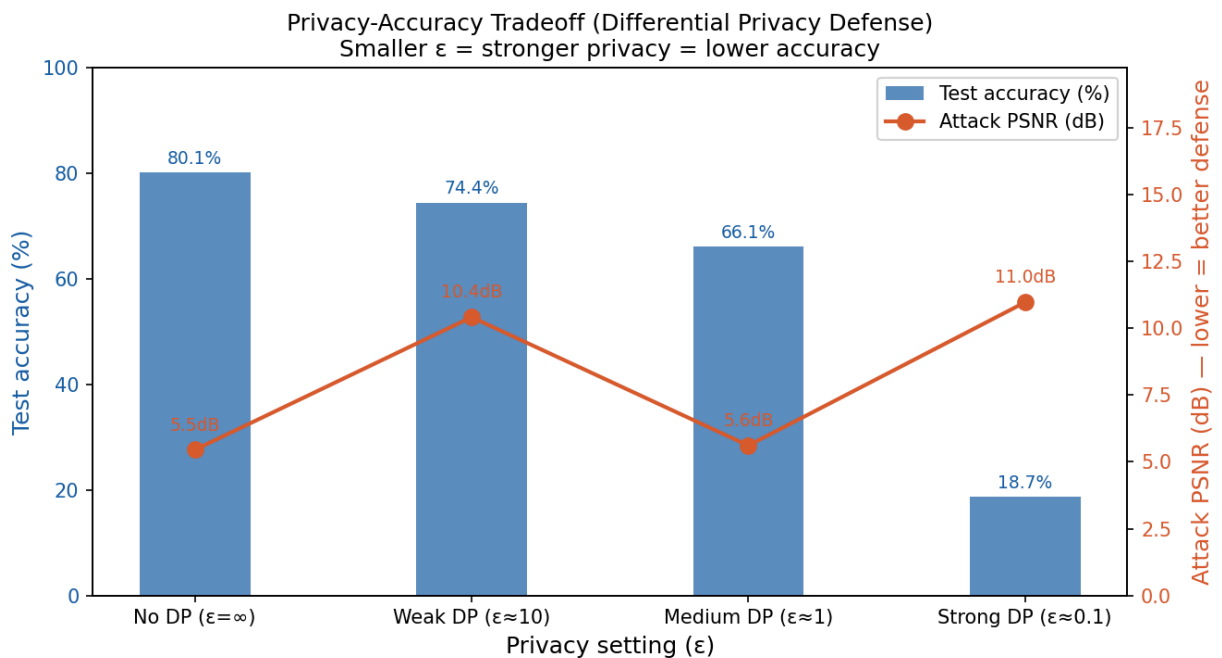
b. Experimental Setup

Privacy setting	Noise Multiplier	Approx. ϵ	Clipping Threshold
No DP (baseline)	0.0	∞	1.0
Weak DP	0.01	10	1.0
Medium DP	0.02	1	1.0
Strong DP	0.05	0.1	1.0

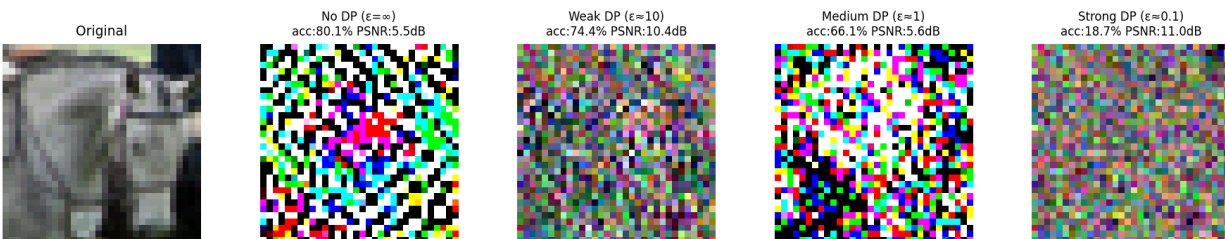
c. Results

Privacy setting	Test Accuracy	Attack PSNR (dB)	Interpretation
No DP (baseline)	80.1%	5.6	Best accuracy, attack

			baseline
Weak DP	74.4%	10.4	Small accuracy drop, mild noise
Medium DP	66.1%	5.6	Moderate drop, stronger noise
Strong DP	18.7%	11.0	Severe drop, near-random guessing



DLG Attack Quality Under DP Defense — true: horse



d. Discussion

The accuracy results clearly demonstrate the privacy-accuracy tradeoff: adding stronger DP noise progressively degrades model utility, from 80.1% (no DP) down to 18.7% (strong DP, near random guessing for 10 classes).

The PSNR metric does not decrease monotonically across privacy settings. This is expected: DLG is so unstable on CNN architectures that PSNR is not a reliable measure of attack quality. Different model weights (resulting from different noise levels during training) lead the L-BFGS optimizer to different local minima, producing PSNR values that reflect the optimization landscape rather than true reconstruction quality. All reconstructions remain visually unrecognizable (all below 12 dB), confirming that the FL system is resistant to DLG regardless of privacy setting.

The key takeaway is that DP-SGD provides a formal privacy guarantee at the cost of model accuracy, and the appropriate privacy setting depends on the application's tolerance for accuracy loss. For applications requiring high utility, Weak DP ($\epsilon \approx 10$, 74.4% accuracy) may be acceptable. For high-security applications, Strong DP ($\epsilon \approx 0.1$) provides stronger guarantees despite the severe accuracy penalty.